M0: EEE3097/8/9S Course Handbook & Project Specifications

Course: EEE3097/8/9S (2026)
Project: Autonomous Micromouse Robotic Maze Solver

---

1. Introduction & Primary Course Task

The UCT Micromouse project is a comprehensive engineering design challenge designed to evaluate and accredit your skills in embedded systems, control theory, and software design.

The Primary Project Task:

Your objective is to design, implement, and validate the code logic that enables a differential-drive robot to autonomously explore a maze, map the layout of its walls, plan the shortest path to a target cell, and execute a high-speed solving sprint.

To pass the course and meet ECSA Graduate Attribute 3 (Design) requirements, you must prove that your mouse uses **active feedback control** to adapt to physical disturbances (such as motor asymmetries and wheel slip) rather than relying on faked or open-loop timed delays.

---

2. Development Tracks & Language Choices

You may choose to implement your algorithms using either of the following two development options. The evaluation criteria are identical for both:

Option A: The Python Track

- Write high-level control code in Python.
- Your scripts run natively on the physical mouse's internal interpreter or interact with the virtual maze simulator using the `uct_mouse` wrapper library.
- *Primary Entry Point:* `python/main.py`.

Option B: The Simulink Track

- Model your algorithms visually using MATLAB, Simulink, and Stateflow.
- Use the Embedded Coder toolbox to compile your models into binary images that run natively on the STM32 processor.
- *Primary Model Template:* `matlab/simulink/StudentTemplate.slx`.

---

3. GA3 Design Project Reports

For each of your two GA3 Design Reports, you must identify and document a structured engineering design process for a chosen subsystem of your choice related to the Micromouse.

Consistent with the professional engineering standard, this selection is completely **open-ended and non-prescriptive**. You may choose any task for which you can confidently present evidence of design thinking.

Illustrative, non-prescriptive examples of design topics include:

- **Stream A (Control & Estimation):** Designing and tuning a discrete PID velocity controller; fusing gyroscope yaw and encoders; or modeling motor parameter identification dynamics.
- **Stream B (Interface & Systems Engineering):** Designing a visual Blockly block library and web-app generator; implementing high-level API safety wrappers; or coding automated hardware self-test calibrators.

---

4. Repository & Workspace Structure

To facilitate updates to the core repository without overwriting your progress, the project workspace is partitioned:

- **Cloning the Workspace:** To clone this repository with all required microcontroller submodules, run this command in your terminal:

  ```
  git clone --recursive https://github.com/nicollsf/UCT-Micromouse.git
  ```

  If you have already cloned the repository without the submodules, initialize them using:

  ```
  git submodule update --init --recursive
  ```

- **Your Sandbox (`/workspace/`):** Put all your Python scripts, custom packages, libraries, and Simulink `.slx` files inside the `/workspace/` directory at the project root. This directory is ignored by Git, meaning your code remains safe and untracked when pulling repository updates.
- **The Deployer Tool (`tools/deploy.py`):** Use this script to copy your local Python files and custom package directories onto the physical mouse's internal drive:

  ```
  python tools/deploy.py --script workspace/my_task/main.py
  ```

- **The Simulator Testbed:** You can test your controller code locally on your laptop before deploying to the physical mouse. Run the visual simulation testbed to evaluate your algorithms against virtual mazes:

  ```
  python tools/physics_sim.py
  ```

- **Simulation Stress-Testing (Perturbations):** To verify that you are using active feedback control (speed matching and gyro heading alignment) rather than hardcoded open-loop delays, the autograder executes your code in co-simulation under randomized perturbations, including:
  - **Motor Asymmetry:** Left/right motor gain offsets (up to $\pm 10\%$).
  - **Wheel Slip / Traction Loss:** Simulated tire slippage to penalize pure time-based dead reckoning.

---

5. Chronological Submissions & Detailed Assessment Rubrics

To satisfy the ECSA Graduate Attribute 3 (Design) accreditation portfolio, you must complete **four primary submissions** in chronological order:

Submission 1: Milestone 1 Code & Demo (25%)

- **Task:** Drive a closed loop: drive 1.0m straight, turn 90° right, and repeat this 4 times to form a 1.0m x 1.0m square, then stop autonomously.
- **Assessment & Grading Metric:** Graded proportionally based on the Euclidean error distance (d_e) from the starting point $(0, 0)$ at the end of the run:
  - **100%:** Excellent feedback control ($d_e \leq 5$ cm).

  - **75%:** Good tracking control (5 cm $< d_e \leq 15$ cm).
  - **60%:** Baseline pass (15 cm $< d_e \leq 30$ cm).
  - *Note:* The autograder applies motor asymmetry ($\pm 10\%$) and wheel slip perturbations in co-simulation to verify active control.

Submission 2: GA3 Design Report 1 (20%)

- **Task:** Submit a formal engineering design report (in PDF format) documenting your closed-loop feedback controller, velocity synchronization, or heading alignment designs from Milestone 1.
- **Template:** Follow the formatting rules and character limits in gareport_guidelines.md and the template in EEE3097_8_9S_designreport.docx.
- **Assessment & Passing Criteria:** Evaluated against the ECSA GA3 Design rubric. Must demonstrate a structured design brief (3.1), alternative evaluations (3.2), and first-principles modeling (3.3).

Submission 3: GA3 Design Report 2 (30%)

- **Task:** A second formal engineering design report (in PDF format) documenting your sensor filters, mapping state flows, routing pathfinders, or visual programming interfaces from Milestone 2.
- **Assessment & Passing Criteria:** Evaluated against the ECSA GA3 Design rubric. Must demonstrate implementation testing (3.4) and critical evaluation (3.5). One resubmission of this report is permitted if required to demonstrate Graduate Attribute competence.

Submission 4: Final Maze Solver Code & Demo (25%)

- **Task:** Navigate a virtual/physical mouse to explore a 4x6 grid maze, map wall configurations, compute the shortest path, and run from start to target at high speed.
- **Assessment & Grading Metric:** Graded proportionally:

$$\text{Score} = 0.5 \cdot (\%\ \text{cells visited}) + 0.5 \cdot (\%\ \text{walls mapped}) - \text{penalties}$$

  - *Collision Tolerance:* Hitting a wall does **not** result in a 0%. The simulation halts, and you receive the score accrued before the collision, minus a -10% penalty.
  - *Bonus:* Extra marks are awarded for successfully planning and executing a high-speed solving run to the target cell.

---

6. Telemetry Logs & Academic Honesty

Your grades are verified through physical run telemetry logs and video evidence:

- **Single-Cable Connection:** Connect the USB cable to the **ST-Link debugger USB port** (the same port used for flashing code). You do not need to swap cables or connect to the processor OTG port.
- **Log Extraction:** Extract the log from your physical mouse by running:

  `python tools/dump_logs.py`

  This saves the output file as `run_log.jsonl` in your folder.
- **Authenticity Verification (Anti-Cheat):**

  - **Device UID:** The log records your microcontroller's unique Device UID. The course convenors check the submitted logs retrospectively. Logs containing identical UIDs under different student accounts are flagged for plagiarism review.
  - **Code FNV-1a Checksum:** The log contains an FNV-1a checksum hash computed in hardware representing the code loaded onto the board. The autograder compiles your submitted script/model locally and verifies that the resulting hash matches the log header. Mismatched hashes will result in an immediate submission rejection.
- **Student Card Video Declaration:** Every validation video must start with a **3-second close-up of your physical Student Card** to serve as your formal academic honesty declaration.